

OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

OptiCrop is a crop recommendation system that uses Machine Learning to help farmers and agricultural users choose the most suitable crop based on soil and weather conditions. The application takes inputs such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall, and recommends the crop that is best suited for those conditions. The system compares different machine learning models and uses the best-performing model to provide accurate crop recommendations. Built with Flask and deployed publicly on Render, OptiCrop provides a simple and user-friendly interface where users can enter their soil and environmental values and receive instant crop recommendations. The frontend is developed using HTML, CSS, JavaScript, and Bootstrap, while the backend processes the user inputs and generates predictions using the trained machine learning model. By combining Machine Learning with web technologies, OptiCrop helps farmers, students, and agricultural professionals make better crop selection decisions.

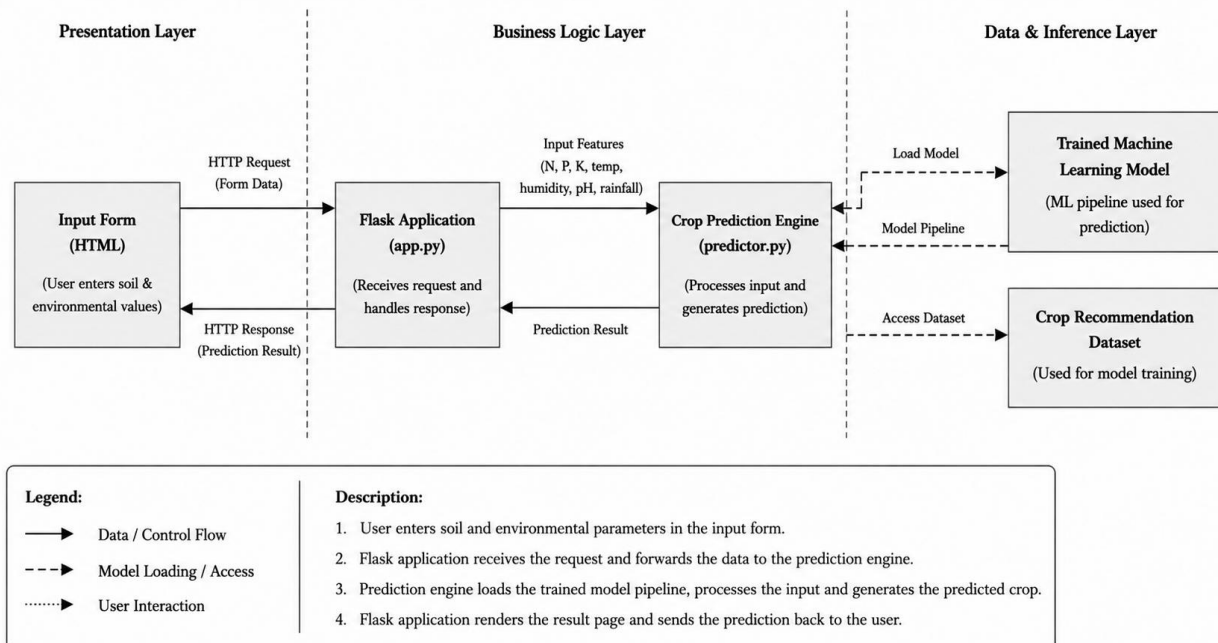
Scenarios

Scenario 1: A farmer enters the soil nutrient values (Nitrogen, Phosphorus, and Potassium) along with temperature, humidity, pH, and rainfall into the application. The system processes the inputs using the trained machine learning model and recommends the most suitable crop for those conditions within a few minutes.

Scenario 2: An agriculture student uses the application to understand how different soil and environmental conditions affect crop selection. By entering different input values, the student can observe the recommended crops and learn about suitable growing conditions for various crops.

Scenario 3: An agricultural officer assists farmers in selecting suitable crops for different fields. By entering the soil test results and weather-related values into OptiCrop, the system provides crop recommendations that help farmers make better cultivation decisions.

Technical Architecture:



Description: OptiCrop follows a modular three-tier architecture to provide accurate and efficient crop recommendations. The frontend provides a responsive user interface developed using HTML, CSS, JavaScript, and Bootstrap, where users can enter soil and environmental parameters. The Flask-based backend receives the user inputs, performs input validation, and forwards the data to the crop prediction engine. The prediction engine loads the trained machine learning model and processes the input values to recommend the most suitable crop. The application is deployed on Render, providing users with an easy-to-use and accessible web-based platform for crop recommendation.

Pre-requisites:

- **Python Programming:** Basic knowledge of Python for developing the machine learning model and backend.
- **Flask Framework:** Familiarity with Flask for building the web application and handling user requests.
- **Machine Learning Fundamentals:** Understanding of machine learning algorithms and the Scikit-learn library.
- **HTML, CSS, JavaScript, and Bootstrap:** Knowledge of frontend technologies for designing a responsive user interface.
- **Data Analysis with Pandas and NumPy:** Basic understanding of handling and preprocessing datasets.
- **Version Control with Git and GitHub:** Knowledge of Git for source code management and collaboration.
- **Development Environment Setup:** Python, pip, virtual environment (venv), and Visual Studio Code or any preferred IDE.
- **Render Deployment:** Basic understanding of deploying Flask applications on Render.

Project Workflow:

Milestone 1: Setup & Data Preparation

- **Activity 1.1:** Initialize the Git repository, configure virtual environment, and set up .gitignore for binary pickles and environments.
- **Activity 1.2:** Download and place the 2,200-row agricultural dataset Crop_recommendation.csv under the data/ folder.
- **Activity 1.3:** Configure dependencies in requirements.txt ensuring generic constraints for modern Python environments.

Milestone 2: Model Comparison & Selection

- **Activity 2.1:** Implement model_training.py and exploration notebook to run Exploratory Data Analysis (EDA) on soil/weather attributes.
- **Activity 2.2:** Train and evaluate Logistic Regression, K-Nearest Neighbors (KNN), Decision Trees, and Random Forests.
- **Activity 2.3:** Save the top-performing Random Forest pipeline (crop_recommendation_model.pkl) containing both scaling and classification layers.

Milestone 3: Core Backend Development

- **Activity 3.1:** Code input range validation and model loading routines inside predictor.py to prevent out-of-bounds inputs.
- **Activity 3.2:** Build Flask application routing in app.py to handle both standard HTML form rendering and REST JSON endpoints.

Milestone 4: Frontend Development

- **Activity 4.1:** Write the user interface templates (home.html, about.html, findyourcrop.html) using semantic markup and Bootstrap 5 grid layout.
- **Activity 4.2:** Design custom visual elements (glassmorphic cards, alternative crop match bars) using vanilla CSS (style.css) and asynchronous AJAX requests (script.js).

Milestone 5: Testing & Performance Verification

- **Activity 5.1:** Implement unit tests (test_system.py) to verify range boundary validation and prediction accuracy
- **Activity 5.2:** Build and execute an automated load testing script (run_perf_test.py) to evaluate latency, throughput, error rate, CPU, and memory limits under concurrent user simulations.

Milestone 6: Conclusion

Milestone 1: Setup & Data Preparation

In this milestone, we focus on configuring the workspace environment, establishing packages, and Analyzing the dataset structure containing 2,200 instances of precision agricultural data.

Activity 1.1: Environment Setup

Start by configuring the virtual environment and installing python dependencies:

```
# Initialize venv
python -m venv venv
venv\Scripts\activate

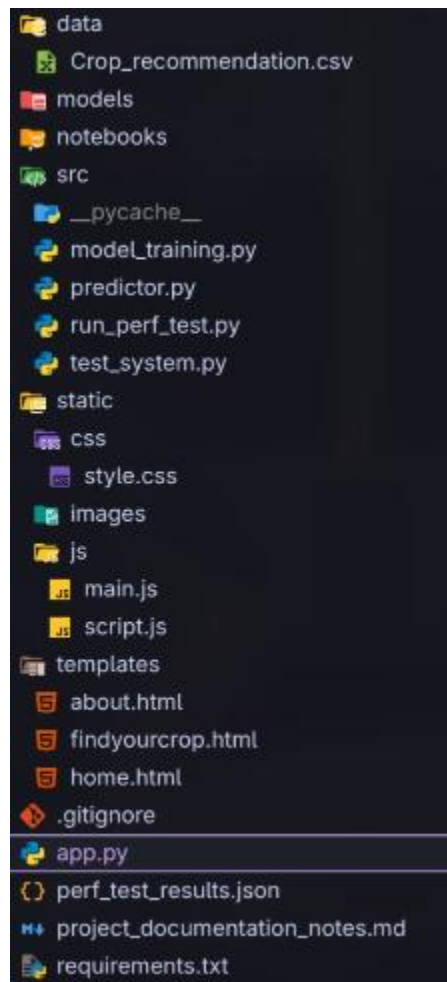
# Install requirements
pip install -r requirements.txt
```

In requirements.txt, verify the target packages:

```
Flask
scikit-learn
pandas
numpy
matplotlib
seaborn
notebook
unicorn
```

Activity 1.2: Check Dataset & Model Directory structure

Verify the data file is placed under the data/ folder and that directories match the project Architecture.



Milestone 2: Model Comparison & Selection

To identify the most accurate classification engine, four algorithms were trained and validated on an 80/20 train/test split.

Model Accuracy & Metric Evaluation

The comparison results printed from `src/model_training.py` indicate:

Machine Learning Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	97.27%	97.40%	97.27%	97.25%
K-Nearest Neighbors	97.95%	98.04%	97.95%	97.93%
Random Forest	99.55%	99.57%	99.55%	99.55%
Decision Tree	97.95%	98.06%	97.95%	97.94%

The best model is a **Random Forest Classifier** with **99.55% accuracy**, which is saved as a serialized pipeline containing a StandardScaler layer to preserve standard scaling during Inference.

```

A optiCrop main v3.14.5
>
19:13:24

A optiCrop main v3.14.5
> python src/model_training.py
19:14:03
Loading dataset from /home/rajan/optiCrop/data/Crop_recommendation.csv...

--- Training and Evaluating Models ---
Logistic Regression -> Accuracy: 0.9727 | Precision: 0.9740 | Recall: 0.9727 | F1-Score: 0.9725
K-Nearest Neighbors -> Accuracy: 0.9795 | Precision: 0.9804 | Recall: 0.9795 | F1-Score: 0.9793
Decision Tree -> Accuracy: 0.9795 | Precision: 0.9806 | Recall: 0.9795 | F1-Score: 0.9794
Random Forest -> Accuracy: 0.9955 | Precision: 0.9957 | Recall: 0.9955 | F1-Score: 0.9955

Best Model: Random Forest with Accuracy: 0.9955

Saved best model pipeline to /home/rajan/optiCrop/models/crop_recommendation_model.pkl

A optiCrop main v3.14.5 1s
> 
19:14:16

```

Milestone 3: Core Backend Development

We create the backend logic to handle data parsing, range validation, and prediction routing.

Activity 3.1: Prediction Engine (src/predictor.py)

Handles model deserialization, validation bounds checks, and predictions. The predict_proba function sorts secondary recommendations to output alternative suitability scores:

```
import os
import pickle
import numpy as np
import pandas as pd

class CropPredictor:
    def __init__(self, model_path=None):
        if model_path is None:
            base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
            model_path = os.path.join(base_dir, "models",
"crop_recommendation_model.pkl")
            self.model_path = model_path
            self._load_model()

    def _load_model(self):
        with open(self.model_path, 'rb') as f:
            self.model = pickle.load(f)

    def validate_inputs(self, n, p, k, temperature, humidity, ph, rainfall):
        errors = {}
        def check_float(val, name, min_val, max_val):
            try:
                f_val = float(val)
                if not (min_val <= f_val <= max_val):
                    errors[name] = f"Must be between {min_val} and {max_val}."
                return f_val
            except (ValueError, TypeError):
                errors[name] = "Must be a valid number."
                return None

        n_val = check_float(n, 'N', 0.0, 200.0)
        p_val = check_float(p, 'P', 0.0, 200.0)
        k_val = check_float(k, 'K', 0.0, 300.0)
        temp_val = check_float(temperature, 'temperature', -10.0, 60.0)
        hum_val = check_float(humidity, 'humidity', 0.0, 100.0)
        ph_val = check_float(ph, 'ph', 0.0, 14.0)
        rain_val = check_float(rainfall, 'rainfall', 0.0, 1000.0)

        if errors:
            return None, errors
```

```

        return [n_val, p_val, k_val, temp_val, hum_val, ph_val, rain_val], None

    def predict(self, features):
        features_df = pd.DataFrame([features], columns=['N', 'P', 'K',
'temperature', 'humidity', 'ph', 'rainfall'])
        return str(self.model.predict(features_df)[0])

    def predict_proba(self, features):
        features_df = pd.DataFrame([features], columns=['N', 'P', 'K',
'temperature', 'humidity', 'ph', 'rainfall'])
        probabilities = self.model.predict_proba(features_df)[0]
        classes = self.model.classes_
        crop_probs = list(zip(classes, probabilities))
        crop_probs.sort(key=lambda x: x[1], reverse=True)
        return crop_probs

```

Activity 3.2: Flask Routing Endpoint (app.py)

Processes standard HTML web forms or incoming JSON requests for remote API consumers:

```

from flask import Flask, render_template, request, jsonify
from predictor import CropPredictor

app = Flask(__name__)
predictor = CropPredictor()

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/predict', methods=['POST'])
def predict():
    is_json = request.is_json
    data = request.get_json() if is_json else request.form

    n = data.get('N')
    p = data.get('P')
    k = data.get('K')
    temperature = data.get('temperature')
    humidity = data.get('humidity')
    ph = data.get('ph')
    rainfall = data.get('rainfall')

    features, errors = predictor.validate_inputs(n, p, k, temperature, humidity, ph,
rainfall)

```



```

if errors:
    return jsonify({"status": "fail", "errors": errors}), 400 if is_json else
render_template('findyourcrop.html', inputs=data, errors=errors)

prediction = predictor.predict(features)
probabilities = predictor.predict_proba(features)
confidence = probabilities[0][1] if probabilities else 1.0

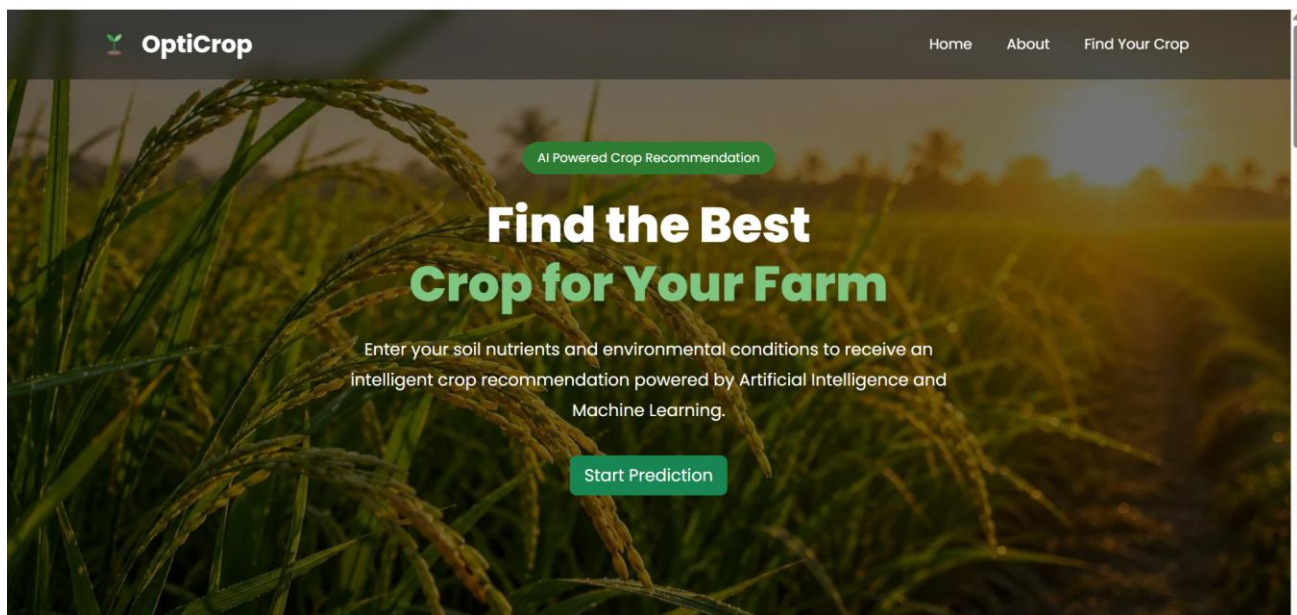
if is_json:
    return jsonify({
        "status": "success",
        "prediction": prediction,
        "probabilities": {crop: float(prob) for crop, prob in probabilities}
    })
    return render_template('findyourcrop.html', prediction=prediction,
confidence=confidence, inputs=data)

```

Milestone 4: Frontend Development

The frontend provides an interactive, accessible dashboard matching the agronomic context.

- **Responsive Input UI:** Forms are styled with clear range helper bounds.
- **Asynchronous Submissions (AJAX):** Using script.js to serialize inputs, POST them to the server, and render result data without forcing full browser page reloads.
- **Secondary Suitability Cards:** Dynamically graphs the top 4 matched crops, showing likelihood percentages so the user can consider backup seeds.



Milestone 5: Testing & Performance Verification

We run two verification passes: unit tests (test_system.py) and a concurrency load script (run_perf_test.py).

Activity 5.1: Unit & Integration Tests

```
A optiCrop main ? v3.14.5 0s
> python "5. Project Development Phase/src/test_system.py"
Model loaded successfully.
.....
[Test] Rice inputs prediction: rice
.
-----
Ran 6 tests in 0.019s

OK
```

Activity 5.2: Concurrency Load & Stress Testing

We simulated 4 scenarios (Sequential, Load, Stress, Spike) simulating up to 100 concurrent Virtual Users (VUs) and 1,650 requests total. The execution results are as follows:

- **Scenario 1 (Homepage Load Test):** 1 VU | 50 Total Reqs | Avg Latency: 0.91 ms | Throughput: 1080.40 req/sec | Error Rate: 0.00%
- **Scenario 2 (Predict API Load Test):** 10 VUs | 100 Total Reqs | Avg Latency: 183.72 ms | Throughput: 53.32 req/sec | Error Rate: 0.00%
- **Scenario 3 (Predict API Stress Test):** 50 VUs | 500 Total Reqs | Avg Latency: 587.93 ms | Throughput: 80.90 req/sec | Error Rate: 0.00%
- **Scenario 4 (Predict API Spike Test):** 100 VUs | 1000 Total Reqs | Avg Latency: 1141.09 ms | Throughput: 83.40 req/sec | Error Rate: 0.00%

Performance Summary:

- **Overall Average Latency: 880.89 ms**
- **Overall Peak Latency: 1290.86 ms**
- **Overall Error Rate: 0.00%**
- **Peak CPU Utilization: 76.20%**
- **Peak Memory Utilization: 1.60%**

```
> source venv/bin/activate 20:07:50

^ optiCrop main ? v3.14.5
> python "5. Project Development Phase/src/run_perf_test.py" 20:07:57
Starting Flask server at /home/rajan/optiCrop/5. Project Development Phase/app.py with /home/rajan/optiCrop/venv/bin/python...
Server response code: 200
Connection successful! Running performance scenarios...

--- Running Scenarios ---

Scenario 1: Homepage Load Test
Virtual Users: 1 | Total Requests: 50
Duration: 0.05 seconds
Avg Response Time: 0.91 ms
Max Response Time: 1.42 ms
Throughput: 1080.40 req/sec
Error Rate: 0.00%

Scenario 2: Predict API Load Test
Virtual Users: 10 | Total Requests: 100
Duration: 1.88 seconds
Avg Response Time: 183.72 ms
Max Response Time: 878.42 ms
Throughput: 53.32 req/sec
Error Rate: 0.00%

Scenario 3: Predict API Stress Test
Virtual Users: 50 | Total Requests: 500
Duration: 6.18 seconds
Avg Response Time: 587.93 ms
Max Response Time: 696.07 ms
Throughput: 80.90 req/sec
Error Rate: 0.00%

Scenario 4: Predict API Spike Test
Virtual Users: 100 | Total Requests: 1000
Duration: 11.99 seconds
Avg Response Time: 1141.09 ms
Max Response Time: 1290.86 ms
Throughput: 83.40 req/sec
Error Rate: 0.00%

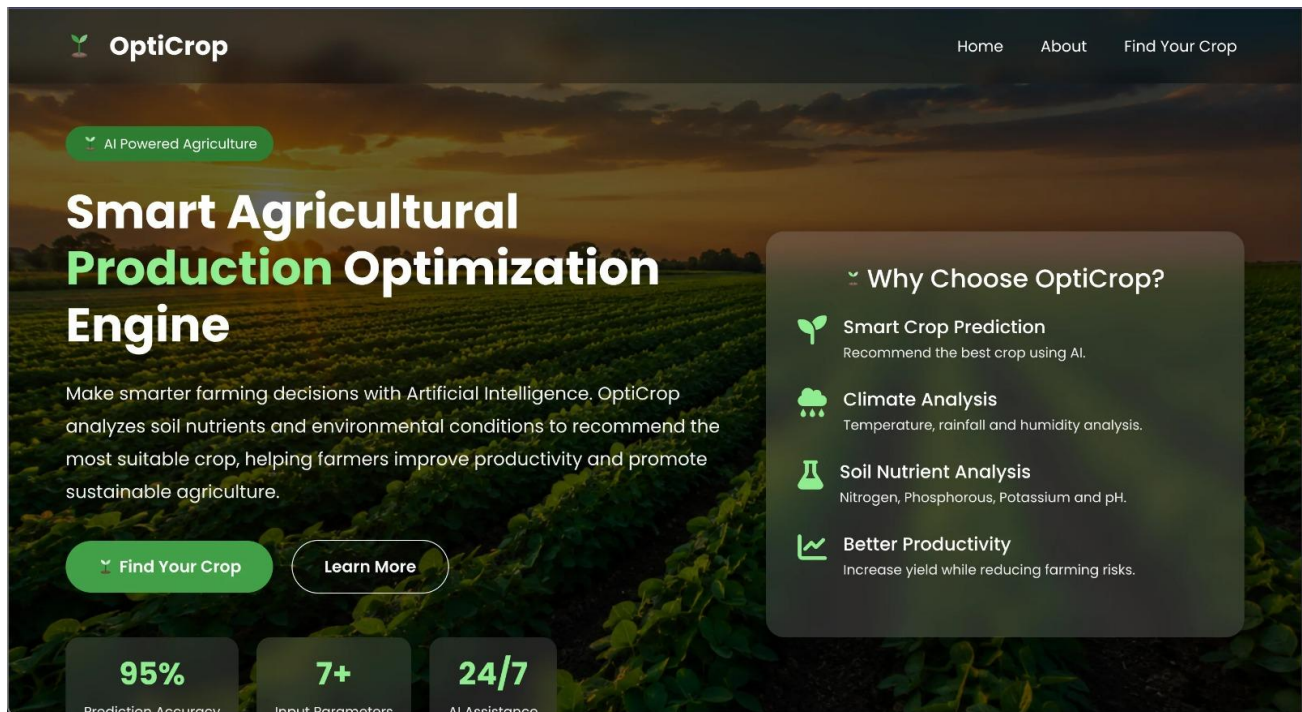
Stopping Flask server...

=== PERFORMANCE TEST SUMMARY ===
Overall Avg Latency: 880.89 ms
Overall Max Latency: 1290.86 ms
Overall Error Rate: 0.00%
Max CPU Observed: 76.20%
Max Memory Observed: 1.60%
Results saved to /home/rajan/optiCrop/5. Project Development Phase/perf_test_results.json
```

Exploring the Website's Web Pages:

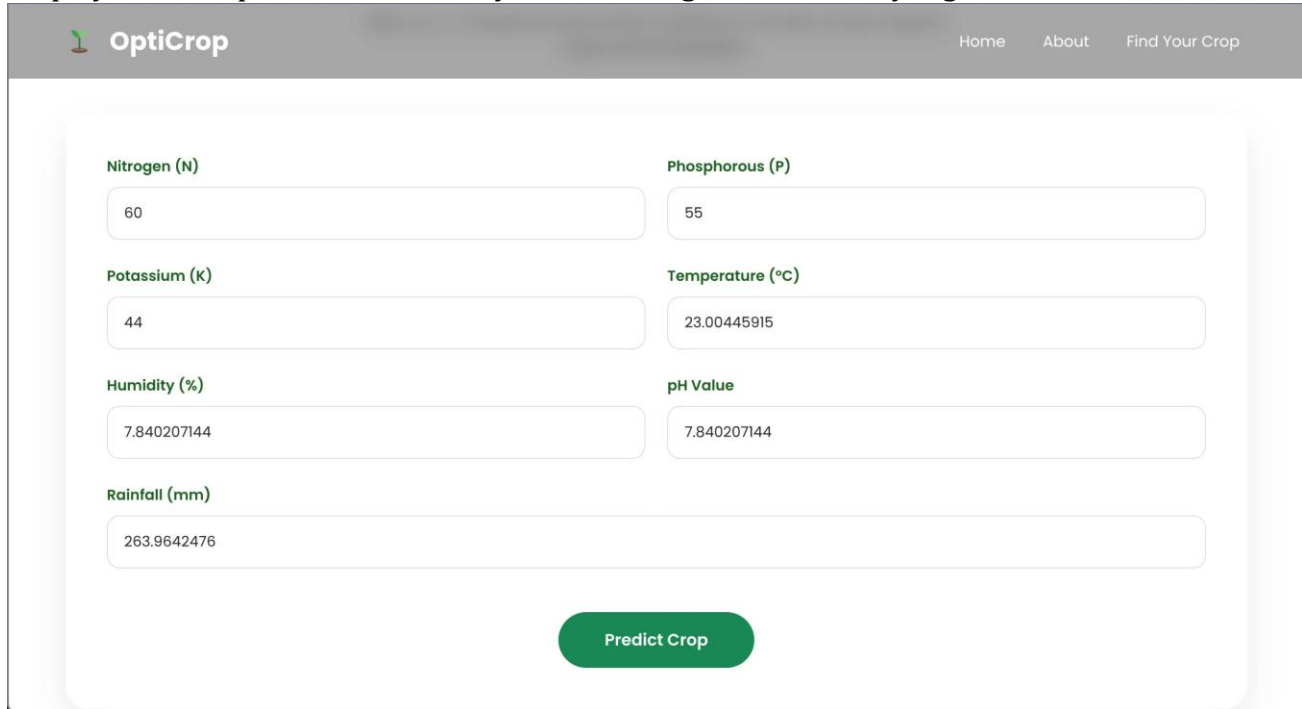
Home / Landing Page:

Exposes introduction banners, features description, and detail cards of standard physical inputs evaluated by the pipeline.



Find Your Crop Page:

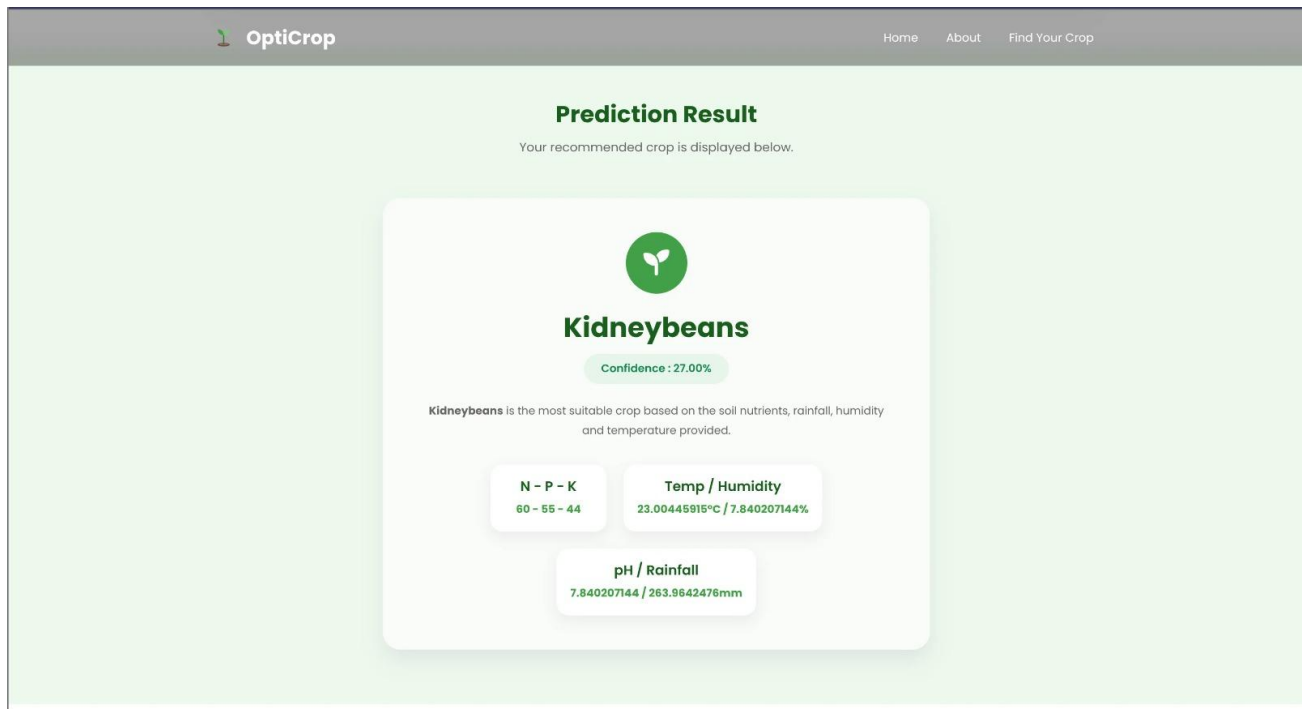
Displays the 7-input numerical entry card utilizing translucent styling and visual cues.



The screenshot shows the 'Find Your Crop' page of the OptiCrop application. The page has a dark grey header with the OptiCrop logo and navigation links: Home, About, and Find Your Crop. The main content area is a light grey card with a white background. It contains seven input fields for numerical data, arranged in two columns. The inputs are: Nitrogen (N) with value 60, Phosphorous (P) with value 55, Potassium (K) with value 44, Temperature (°C) with value 23.00445915, Humidity (%) with value 7.840207144, pH Value with value 7.840207144, and Rainfall (mm) with value 263.9642476. A green 'Predict Crop' button is centered at the bottom of the card.

Recommendation Result Page:

Renders the crop result name, confidence match, and progress indicators showing secondary alternative recommendations.



The screenshot shows the 'Prediction Result' page of the OptiCrop application. The page has a dark grey header with the OptiCrop logo and navigation links: Home, About, and Find Your Crop. The main content area is a light green background. At the top, it says 'Prediction Result' and 'Your recommended crop is displayed below.' Below this is a white card with a green plant icon and the text 'Kidneybeans'. A green bar indicates 'Confidence : 27.00%'. Below the confidence bar, it says 'Kidneybeans is the most suitable crop based on the soil nutrients, rainfall, humidity and temperature provided.' At the bottom of the card, there are three boxes showing the input values: 'N - P - K' with values 60 - 55 - 44, 'Temp / Humidity' with values 23.00445915°C / 7.840207144%, and 'pH / Rainfall' with values 7.840207144 / 263.9642476mm.

Conclusion:

OptiCrop is a machine learning-based crop recommendation system developed using Flask with a responsive web interface built using HTML, CSS, JavaScript, and Bootstrap. The application analyzes soil and environmental parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall to recommend the most suitable crop. The project includes dataset preparation, model training, web application development, and deployment on Render, demonstrating how machine learning can support better crop selection.

In the future, OptiCrop can be enhanced by integrating real-time weather data, fertilizer recommendations, multilingual support, and IoT-based soil sensors. Additional features such as user accounts and prediction history can further improve the application, making it a more useful and complete solution for smart agriculture.